Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Examples: Using the API to Download Sensor Images

You can use the Stellar Cyber API to download sensor images from the Stellar Cyber Platform. Using the API to download sensor images has the following benefits:

- Reduces the load on the Stellar Cyber GUI server and associated network infrastructure.
- Allows deployment of server sensors in customized automation scripts.

Refer to **Configuring API Authentication** for general requirements to use the API.

API Endpoints for Sensor Image Downloads

The API provides the following endpoints for use in downloading sensor images:

- Retrieve information on the sensor images available for download:

  `https://<Platform URL>/connect/api/v1/sensor-images`

- Download a specified image:

  `https://<Platform URL>/connect/api/v1/sensor-images/download/<version>/<image>`

Syntax Details

The reference material from the API's swagger.json file provides details on the syntax for both of the API endpoints listed above. In addition, you can use the **? > API Docs** link to try out specific commands directly from your browser. Click the Authorize button at the top of the page that appears, supply the token from the **System | ORGANIZATION MANAGEMENT | Users** page for your account, and you can try out public API commands.

Examples

This section provides some Python 3 scripts that you can adapt for your own needs. In all cases, you will need to perform the following changes:

- Replace the value `HOST = "<Platform Hostname or IP>"` with the hostname or IP address of your Stellar Cyber Platform.
- Replace the value `email = "<email>"` with the email address for a valid account on your own Stellar Cyber Platform.
- Replace the value `refresh_token = "<token>"` with the actual API token for the specified user account from the **System | ORGANIZATION MANAGEMENT | Users** page.

Retrieve and Display Information on Available Images

The first script simply prints the available images to the screen:

```
#!/usr/bin/python3

import base64
from urllib.parse import urlunparse
import hashlib
from tqdm import tqdm

import requests

requests.packages.urllib3.disable_warnings()

# Hostname or IP for our Stellar Cyber Platform
HOST = "<Platform Hostname or IP>"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

def getSensorImagesInfo(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/sensor-images", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    return res.json()

if __name__ == "__main__":
    # step1: generate refresh token via UI
    #  --> done
    refresh_token = "<token>"

    # step2: generate access token (JWT)
    jwt = getAccessToken(refresh_token)
    # print(jwt)

    # step3: get info
    res = getSensorImagesInfo(jwt)
    print(res)
```

Sample Output

Click here to see sample output from the previous script, formatted for ease of reading. As you can see, there are five different images available for download:


```
{'data': {'datasensor': {

'upgrade-linux_saas': {'display_value': 'Suse', 'item_list': ['Suse 12', 'Suse
15'], 'name': 'upgrade-linux_saas', 'package': 'ds_linux_install_all_in_one.sh',
'sha1': '6212787e2854ac7d42a67dc1307845d65f0465e3', 'size': 325425537, 'version':
'5.4.0', 'download_path': '/connect/api/v1/sensor-
images/download/5.4.0/ds_linux_install_all_in_one.sh'},
```

'upgrade-modular-ova_saas': {'display_value': 'VMWare ESXi', 'name': 'upgrade-modular-ova_saas', 'package': 'aella-modular-ds-5.4.0.ova', 'sha1': '85862339d36944e6080d705a[illegible]87e17d7ca086', 'size': 1510031360, 'version': '5.4.0', 'download_path': '/connect/api/v1/sensor-images/download/5.4.0/aella-modular-ds-5.4.0.ova'},

'upgrade-modular-qcow2_saas': {'display_value': 'KVM', 'name': 'upgrade-modular-qcow2_saas', 'package': 'aella-modular-ds-5.4.0.qcow2.zip', 'sha1': 'e5a7cd3257a71babbd386077128834ef7a03c27e', 'size': 1551359552, 'version': '5.4.0', 'download_path': '/connect/api/v1/sensor-images/download/5.4.0/aella-modular-ds-5.4.0.qcow2.zip'},

'upgrade-modular-vhdx_saas': {'display_value': 'Hyper-V', 'name': 'upgrade-modular-vhdx_saas', 'package': 'aella-modular-ds-5.4.0.vhdx', 'sha1': '6d5740e9c6ca8c883a1509b5acb0051e5da22f3c', 'size': 4169138176, 'version': '5.4.0', 'download_path': '/connect/api/v1/sensor-images/download/5.4.0/aella-modular-ds-5.4.0.vhdx'},

'upgrade-windows-x64': {'display_value': 'Windows Server Sensor', 'item_list': ['Windows 2008 R2', 'Windows 2012', 'Windows 2016', 'Windows 2019'], 'name': 'upgrade-windows-x64', 'package': 'aellads_5.4.0_windows-x64_20231216_b345015.msi', 'sha1': '22ae7473d2fcb9044234f0a7dd8dec1ee3ce5c7f', 'size': 155331584, 'version': '5.4.0', 'download_path': '/connect/api/v1/sensor-images/download/5.4.0/aellads_5.4.0_windows-x64_20231216_b345015.msi'}}}}

Download Images

The second script downloads images and calculates and compares their hash values to make sure they downloaded successfully. The files are stored in **/tmp/stellar-images/download/**. You can change this path by editing the script.

```
#!/usr/bin/python3

import base64
from urllib.parse import urlunparse
import hashlib
from tqdm import tqdm

import requests

requests.packages.urllib3.disable_warnings()

# Hostname or IP for our Stellar Cyber Platform
HOST = "<Platform Hostname or IP>"


def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]


def getSensorImagesInfo(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/sensor-images", "", "", ""))
    res = requests.get(url, headers=headers, verify=False)
    return res.json()


def download_sensor_image(token, path, destination, expected_sha1):
    headers = {"Authorization": "Bearer " + token}
```

```
    url = urlunparse(("https", HOST, path, "", "", ""))
    response = requests.get(url, headers=headers, verify=False, stream=True)

    if response.status_code == 200:
        total_size = int(response.headers.get("Content-Length", 0))
        with open(destination, "wb") as file, tqdm(total=total_size, unit="B",
unit_scale=True) as pbar:
            for data in response.iter_content(chunk_size=1024):
                pbar.update(len(data))
                file.write(data)

        calculated_sha1 = calculate_sha1(destination)
        print(f"Sha1 of the downloaded file: {calculated_sha1}")
        print(f"Sha1 of the expected     : {expected_sha1}")

        if calculated_sha1 == expected_sha1:
            print("File downloaded and verified successfully.")
        else:
            print("Verification failed. The downloaded file may be corrupted.")
    else:
        print(f"Download failed. Status code: {response.status_code}")


def calculate_sha1(file_path):
    sha1 = hashlib.sha1()
    with open(file_path, "rb") as file:
        for byte_block in iter(lambda: file.read(1024 * 1024), b""):
            sha1.update(byte_block)
    return sha1.hexdigest()


if __name__ == "__main__":
    # step1: generate refresh token via UI
    #  --> done
    refresh_token = "<token>"

    # step2: generate access token (JWT)
    jwt = getAccessToken(email, refresh_token)
    # print(jwt)

    # step3: get info
    res = getSensorImagesInfo(jwt)
    # print(res)

    # step4: get
    download_path = res["data"]["datasensor"]["upgrade-linux_saas"]["download_path"]
    expected_sha1 = res["data"]["datasensor"]["upgrade-linux_saas"]["sha1"]
    file_name = res["data"]["datasensor"]["upgrade-linux_saas"]["package"]
    destination = f"/tmp/stellar-images/download/{file_name}"
    print(f"Download file from: {download_path} to ")
    download_sensor_image(jwt, download_path, destination, expected_sha1)
```


© 2026 Stellar Cyber . All rights reserved.